AI Paper Solution

Q.1

(a) 1-mark shorts (3 × 1)

1. **Rote learning (in AI):** Storing and reproducing exact mappings (input→output) without deriving general rules; essentially "memorization."
2. **Goal of a planning system:** To find a sequence of actions that transforms the initial state into a desired goal state while respecting preconditions and effects.
3. **Range of a membership function (fuzzy logic):** .

(b) Objective/MCQ/TF/Fill (7 × 1)

1. **Learning MOST evident in Google Duplex when trained on large recorded conversations: (C) Learning from Examples** (supervised/inductive from data).
2. **Alpha–beta pruning always changes the final minimax value. False.** It only prunes branches that cannot affect the backed-up value; the final minimax decision remains the same.
3. **Common game-playing search algorithm: Minimax** (typically with **alpha–beta pruning**).
4. **Converting fuzzy values to a crisp output is called: Defuzzification.**
5. **Benefit of learning by taking advice (e.g., Duplex):** Incorporates teacher/user guidance to correct responses, reduces exploration, speeds learning, and improves appropriateness/safety of replies.
6. **Fuzzification (in control systems):** Converting crisp sensor inputs into degrees of membership over fuzzy sets (e.g., "Cold," "Hot").
7. **How linguistic variables help:** They capture vagueness with human terms ("hot," "high"), enabling simple, interpretable rules and smoother reasoning over uncertainty.

Q.2 (10 marks)

(a) Two questions of 2 marks (2 × 2)

1. **Inductive learning → better Duplex conversations:** Train on labeled dialogs to generalize mappings from context → next utterance/act; models infer patterns like

prosody, timing, confirmations, and repair strategies, improving naturalness and task success.

2. **Why membership functions matter:** They map crisp inputs to fuzzy degrees, enabling partial truth, overlap between concepts, smooth rule firing, and tunable sensitivity—core to fuzzy inference.

(b) Two questions of 3 marks (2 × 3)

1. **Fuzzification vs. Defuzzification (with example):**
    - *Fuzzification:* C → , .
    - *Rule firing:* IF Hot THEN Fan High; IF Comfortable THEN Fan Medium.
    - *Defuzzification:* Aggregate outputs, then compute a crisp fan speed (e.g., centroid) → say **64%** of max speed.
      Difference: Fuzzification maps crisp→fuzzy; defuzzification maps aggregated fuzzy control action → single crisp actuator command.
2. **Alpha–beta in tic-tac-toe:** It prunes branches that cannot influence the root decision given current (best for maximizer) and (best for minimizer). This reduces node expansions dramatically (from toward with good ordering). **Unchanged:** The **optimal minimax value and move** remain exactly the same as full minimax.

Q.3 Attempt any TWO (5 marks each)

(i) Goal-Stack Planning (Blocks World): Move A from on B to the table

- **Operators (STRIPS-style):**
    - Unstack(x, y): Pre: On(x,y), Clear(x), HandEmpty. Add: Holding(x), Clear(y). Del: On(x,y), HandEmpty.
    - PutDown(x): Pre: Holding(x). Add: On(x,Table), Clear(x), HandEmpty. Del: Holding(x).
- **Goal:** On(A,Table).
- **Initial (relevant) facts:** On(A,B), Clear(A), On(B,Table), HandEmpty, Clear(TableSpot).
- **Goal stack evolution (sketch):**

1. Push On(A,Table) → choose PutDown(A) → push Holding(A).

2. To get Holding(A), choose Unstack(A,B) → push On(A,B), Clear(A), HandEmpty (all true), execute Unstack(A,B).

3. Now Holding(A) true → execute PutDown(A).

- **Plan (sequence):** Unstack(A,B) → PutDown(A).
  Result: A is on the table; B becomes Clear.

(ii) Learning types in Google Duplex

- **Rote Learning:** Cache frequent phrases/slot values (e.g., "I'd like a table for two at 7 pm") for quick retrieval.
- **Learning from Examples:** Supervised training on large dialog corpora to map contexts to next actions/words.
- **Learning by Taking Advice:** Human feedback/reviews and policy shaping ("avoid interrupting," "confirm name") refine behavior.
- **Learning in Problem Solving:** Planning dialog acts to satisfy task constraints (check availability → propose alternatives → confirm), optimizing success under constraints (time windows, user preferences).

(iii) Fuzzy rule-based control in practice (evaluate)

- **Use cases:** Washing machines (soil level→wash time), HVAC (temp/humidity→fan/AC), camera autofocus (contrast→step size).
- **Advantages:** Handles imprecision; interpretable rules; smooth control without exact models; robust to sensor noise.
- **Limitations:** Rule base tuning is manual; scaling to many variables explodes rules; lacks long-horizon optimality; can be outperformed by model-based or learning controllers when good models/data exist.

Q.4 (10 marks)

(a) Fuzzy AC controller (Temp & Humidity → Fan Speed)

- **Inputs:** Temperature ; Humidity .
  Output: FanSpeed .
- **Rule examples (Mamdani):**
  1. IF Hot AND High → Fan = High.
  2. IF Hot AND Medium → High.
  3. IF Hot AND Low → Medium/High (often High to quickly cool).
  4. IF Comfortable AND High → Medium.
  5. IF Comfortable AND Medium → Medium.

6. IF Comfortable AND Low → Low.
7. IF Cold (any) → Low.

- **Case asked: Hot**, **High**.
    - **Fuzzification:** Compute and (e.g., 0.8 and 0.7).
    - **Inference:** Rule 1 fires with strength → clip "Fan=High" at 0.7. Other Hot rules may also contribute; Comfortable/Cold rules likely negligible.
    - **Aggregation:** Combine all fired outputs (max of clipped sets).
    - **Defuzzification (centroid):** Weighted center over Low/Med/High curves yields a crisp speed near the **High** region (e.g., ~80–90% of max).
      Final action: Set **High fan speed** (or a crisp value close to the High peak).

(b) Minimax vs. Alpha–Beta (analysis)

- **Minimax:** Exhaustively explores game tree to depth , backing up values assuming optimal opponents. Complexity .
- **Alpha–Beta:** Same optimal decision but prunes branches that cannot improve the current / bounds.
    - **Best-case (good move ordering):** Effective branching ~; much deeper lookahead for the same time.
    - **Trade-off:** Alpha–beta reduces **computation time** per depth, which can be reinvested to **increase search depth**, improving play strength without changing optimality at a given depth.
- **Conclusion:** Use alpha–beta with strong ordering (history/iterative deepening) to keep optimal choices while achieving far greater depth.

OR

(b-OR) Goal-stack plan: set a dinner table (plate, cup, spoon)

- **Top goals:** OnTable(Plate) & OnTable(Cup) & OnTable(Spoon), and each at its correct **place** (e.g., Plate center, Cup top-right, Spoon right of plate).
- **Subgoals (per item):** Clear(PlaceX), Holding(ItemX) → PutDownAt(ItemX, PlaceX).
- **Actions (examples):**
    - PickUp(x) (Pre: On(x,Counter), Clear(x), HandEmpty).
    - PutDownAt(x, p) (Pre: Holding(x), Clear(p)).
- **Planner sequence (one feasible order):**

1. PickUp(Plate) → PutDownAt(Plate, Center).
2. PickUp(Cup) → PutDownAt(Cup, TopRight).
3. PickUp(Spoon) → PutDownAt(Spoon, RightOfPlate).

Preconditions ensure places are clear; HandEmpty maintained between items.
Result: Table set with required placements.